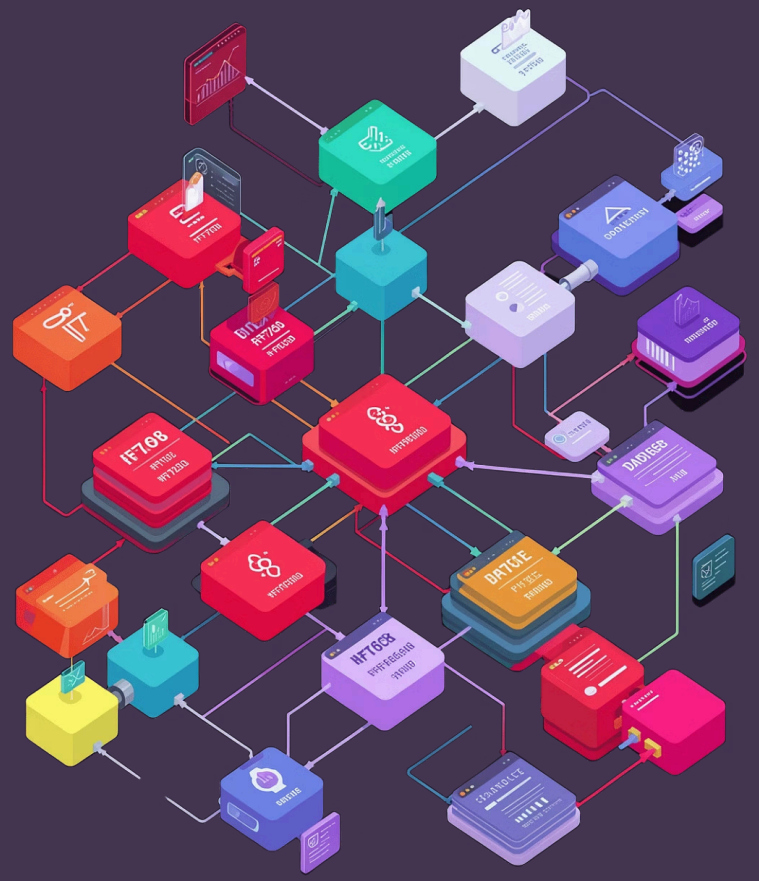
Algoritmos de Clasificación en Machine Learning

La clasificación es una de las tareas fundamentales del aprendizaje supervisado. Su objetivo es predecir a qué **categoría o clase** pertenece una nueva observación, basándose en ejemplos previamente etiquetados. Desde detectar correos no deseados hasta diagnosticar enfermedades, la clasificación está presente en innumerables aplicaciones del mundo real.

MACHINE LEARNING | APRENDIZAJE SUPERVISADO | CLASIFICACIÓN



Made with GAMMA

¿Qué significa clasificar?

La idea central

Clasificar consiste en asignar una **etiqueta discreta** a una entrada. A diferencia de la regresión —donde predecimos un valor continuo como el precio de una vivienda—, en clasificación la salida es siempre una **categoría**: spam o no spam, enfermo o sano, gato o perro.

El modelo aprende patrones a partir de datos de entrenamiento ya etiquetados y, una vez entrenado, es capaz de generalizar y asignar la clase correcta a nuevas observaciones que nunca ha visto antes.

¿Qué vamos a predecir?

La variable objetivo en clasificación es siempre **discreta y categórica**. Esto significa que el conjunto de posibles respuestas es finito y conocido de antemano.

- **Binaria:** solo dos clases posibles (ej. positivo/negativo, sí/no)
- **Multiclase:** tres o más clases (ej. tipo de flor, dígito manuscrito)
- **Multilabel:** una instancia puede pertenecer a varias clases simultáneamente

El modelo no predice un número cualquiera, sino que estima la **probabilidad** de pertenencia a cada clase y asigna la de mayor probabilidad.

Made with GAMMA

Clasificación Binaria vs. Multiclase

🔵 Clasificación Binaria

La forma más sencilla de clasificación. El modelo decide entre **exactamente dos clases**. Es el punto de entrada natural para entender los algoritmos de clasificación.

- **Ejemplos:** Spam / No spam, Fraude / Legítimo, Enfermo / Sano
- **Salida típica:** Probabilidad entre 0 y 1 (usando sigmoide)
- **Umbral de decisión:** habitualmente 0.5, pero ajustable
- **Algoritmos comunes:** Regresión logística, SVM, Árboles de decisión

🟢 Clasificación Multiclase

Cuando hay **más de dos categorías** posibles, el problema se vuelve multiclase. El modelo debe discriminar entre múltiples opciones simultáneamente.

- **Ejemplos:** Reconocimiento de dígitos (0-9), Especies de flores, Sentimiento (positivo, neutro, negativo)
- **Salida típica:** Vector de probabilidades que suma 1 (usando softmax)
- **Estrategias:** One-vs-Rest, One-vs-One, o nativo en algunos algoritmos
- **Algoritmos comunes:** Random Forest, Redes Neuronales, SVM multiclase

ⓘ 💡 **Nota importante:** Muchos algoritmos binarios se pueden extender a multiclase mediante estrategias como **One-vs-Rest (OvR)**, donde se entrena un clasificador por cada clase frente al resto, o **One-vs-One (OvO)**, donde se entrena un clasificador para cada par de clases.

Made with GAMMA

La Importancia del Balanceo de Clases

¿Qué es el desbalanceo?

Un conjunto de datos está **desbalanceado** cuando las clases no están representadas de forma equitativa. Por ejemplo, si el 95 % de los correos son legítimos y solo el 5 % son spam, el modelo podría aprender a predecir siempre "legítimo" y alcanzar un 95 % de precisión... ¡sin haber aprendido nada útil!

El desbalanceo es muy común en la práctica: fraudes bancarios (menos del 1 %), diagnósticos de enfermedades raras, detección de fallos en maquinaria industrial.

Consecuencias del desbalanceo

- El modelo se sesga hacia la clase mayoritaria
- Las métricas como *accuracy* pueden ser engañosas
- La clase minoritaria (a menudo la más importante) se ignora

¿Cómo solucionarlo?

Oversampling

Replicar o generar sintéticamente ejemplos de la clase minoritaria. Técnica popular: **SMOTE** (Synthetic Minority Over-sampling Technique).

Undersampling

Reducir el número de ejemplos de la clase mayoritaria para equilibrar el conjunto. Útil cuando hay muchos datos disponibles.

Pesos de clase

Asignar mayor peso a los errores en la clase minoritaria durante el entrenamiento. Muchos algoritmos lo soportan nativamente.

Métricas adecuadas

Usar **Precisión, Recall, F1-Score** o **AUC-ROC** en lugar de accuracy simple para evaluar correctamente el modelo.

Made with GAMMA

¿Cómo medimos el rendimiento? Accuracy y Matriz de Confusión

Evaluar un clasificador va mucho más allá de calcular el porcentaje de aciertos. Necesitamos herramientas que nos digan **qué tipo de errores** comete el modelo y en qué clases falla más.

Accuracy (Precisión Global)

El **accuracy** es la métrica más intuitiva: proporción de predicciones correctas sobre el total.

$$\text{Accuracy} = \frac{\text{Predicciones Correctas}}{\text{Total de Predicciones}}$$

Sin embargo, el accuracy puede ser **muy engañoso** en conjuntos desbalanceados. Si el 99 % de las transacciones son legítimas, un modelo que siempre prediga "legítimo" tendrá un 99 % de accuracy pero será completamente inútil para detectar fraudes.

⚠ ⚠️ **Regla de oro:** Nunca uses solo accuracy cuando las clases estén desbalanceadas. Complementa siempre con la matriz de confusión y métricas derivadas.

La Matriz de Confusión

La **matriz de confusión** es una tabla que muestra el desglose completo de las predicciones del modelo, comparando las clases reales con las predichas. Para clasificación binaria:

	Predicho Positivo	**Predicho Negativo**
Real Positivo	✅ VP (Verdadero Positivo)	❌ FN (Falso Negativo)
Real Negativo	❌ FP (Falso Positivo)	✅ VN (Verdadero Negativo)

- **VP:** Correctamente identificado como positivo
- **VN:** Correctamente identificado como negativo
- **FP:** Error tipo I — alarmas falsas
- **FN:** Error tipo II — casos perdidos (a menudo el más crítico)

Made with GAMMA

Leyendo la Matriz de Confusión: Visualización y Métricas Derivadas

Una vez construida la matriz de confusión, podemos extraer métricas mucho más informativas que el accuracy simple. Cada métrica responde a una pregunta diferente sobre el comportamiento del modelo.

Precisión (Precision)

De los que el modelo predijo como positivos, ¿cuántos lo eran realmente?

$$P = \frac{VP}{VP + FP}$$

Alta precisión = pocas alarmas falsas. Importante cuando los falsos positivos son costosos.

Exhaustividad (Recall)

De los positivos reales, ¿cuántos detectó el modelo?

$$R = \frac{VP}{VP + FN}$$

Alto recall = pocos casos perdidos. Crítico en diagnóstico médico o detección de fraudes.

F1-Score

Media armónica entre precisión y recall. Equilibra ambas métricas en un solo número.

$$F1 = 2 \cdot \frac{P \cdot R}{P + R}$$

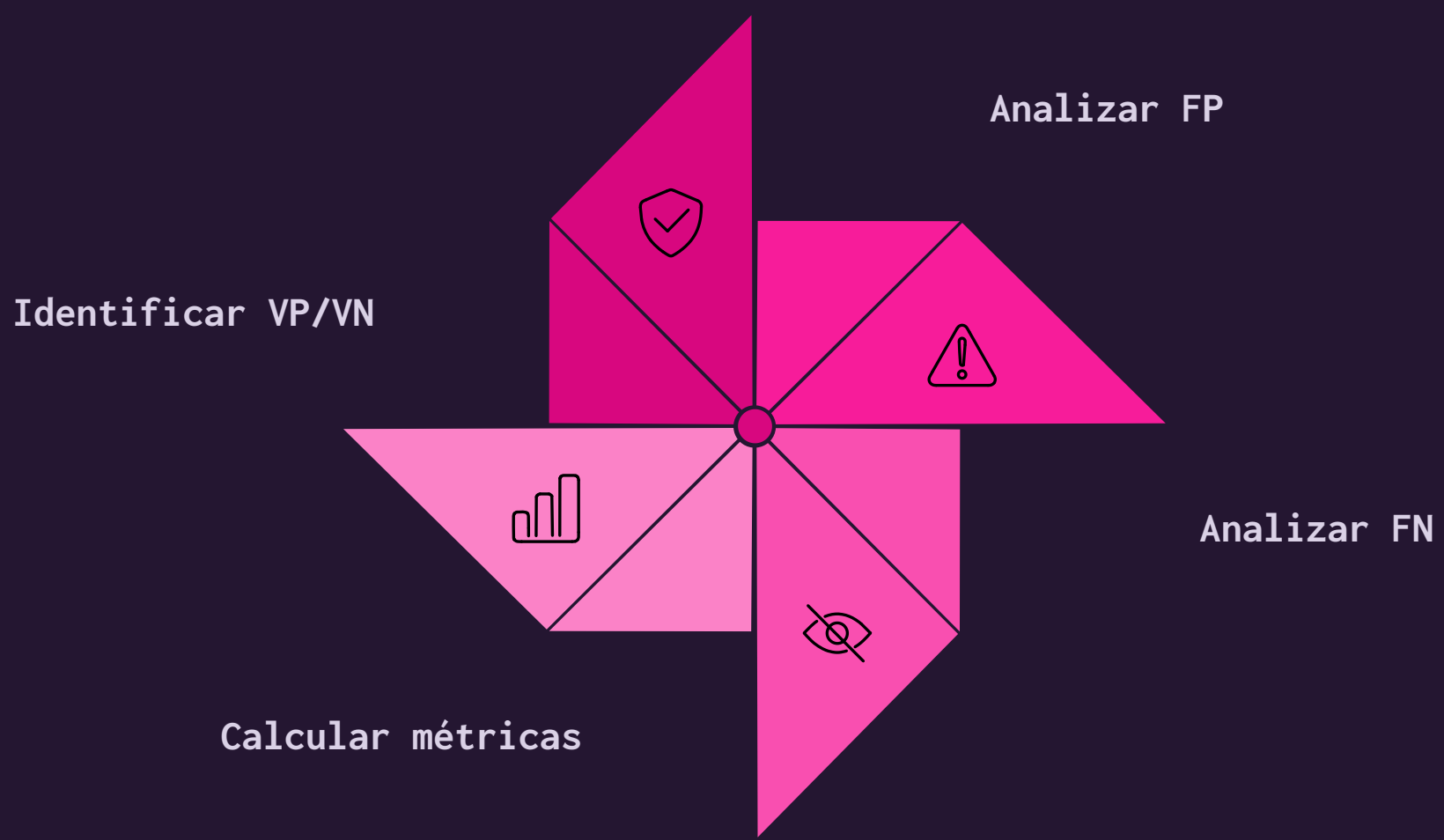
Útil cuando hay desbalanceo y necesitas un único indicador de rendimiento.

AUC-ROC

Área bajo la curva ROC. Mide la capacidad del modelo para discriminar entre clases a distintos umbrales.

$$\text{AUC} \in [0, 1]$$

1.0 = clasificador perfecto; 0.5 = equivalente a adivinar al azar.



El diagrama anterior resume el flujo de interpretación: comenzar identificando los aciertos del modelo (VP y VN), luego examinar los dos tipos de error por separado, y finalmente sintetizar el rendimiento con métricas derivadas que equilibren ambos tipos de error según el contexto del problema.

Made with GAMMA

Modelo 1: Regresión Logística

¿Qué es?

A pesar de su nombre, la **regresión logística** es un algoritmo de clasificación, no de regresión. Modela la probabilidad de que una instancia pertenezca a una clase usando la **función sigmoide**, que transforma cualquier valor real en una probabilidad entre 0 y 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Donde z es una combinación lineal de las características de entrada. Si $\sigma(z) \geq 0.5$, se predice la clase positiva; en caso contrario, la negativa.

Características clave

- **Lineal:** crea fronteras de decisión lineales en el espacio de características
- **Interpretable:** los coeficientes indican la importancia de cada variable
- **Rápido:** muy eficiente en conjuntos de datos grandes
- **Probabilístico:** devuelve probabilidades, no solo etiquetas

Cuándo usarlo

→ Como **línea base (baseline)** en cualquier problema de clasificación binaria

→ Cuando necesitas **interpretabilidad** y explicabilidad del modelo

→ En problemas donde la relación entre características y clase es **aproximadamente lineal**

→ Como capa final en redes neuronales para clasificación

📊 **Extensión multiclase:** La regresión logística se extiende a múltiples clases mediante **regresión softmax**, que generaliza la sigmoide a K clases.

Made with GAMMA

Modelo 2: Árboles de Clasificación

¿Cómo funcionan?

Un **árbol de decisión** divide recursivamente el espacio de características mediante preguntas del tipo "¿Es X mayor que umbral?". Cada división busca maximizar la **pureza** de los nodos resultantes, medida comúnmente con el índice Gini o la entropía.

- **Nodo raíz:** la primera pregunta, la que más separa las clases
- **Nodos internos:** preguntas intermedias que refinan la clasificación
- **Hojas:** nodos terminales que asignan una clase definitiva

El árbol "aprende" seleccionando en cada paso la característica y el umbral que mejor separan las clases. Es un proceso **codicioso (greedy)**: toma la mejor decisión local en cada paso.

Ventajas y limitaciones

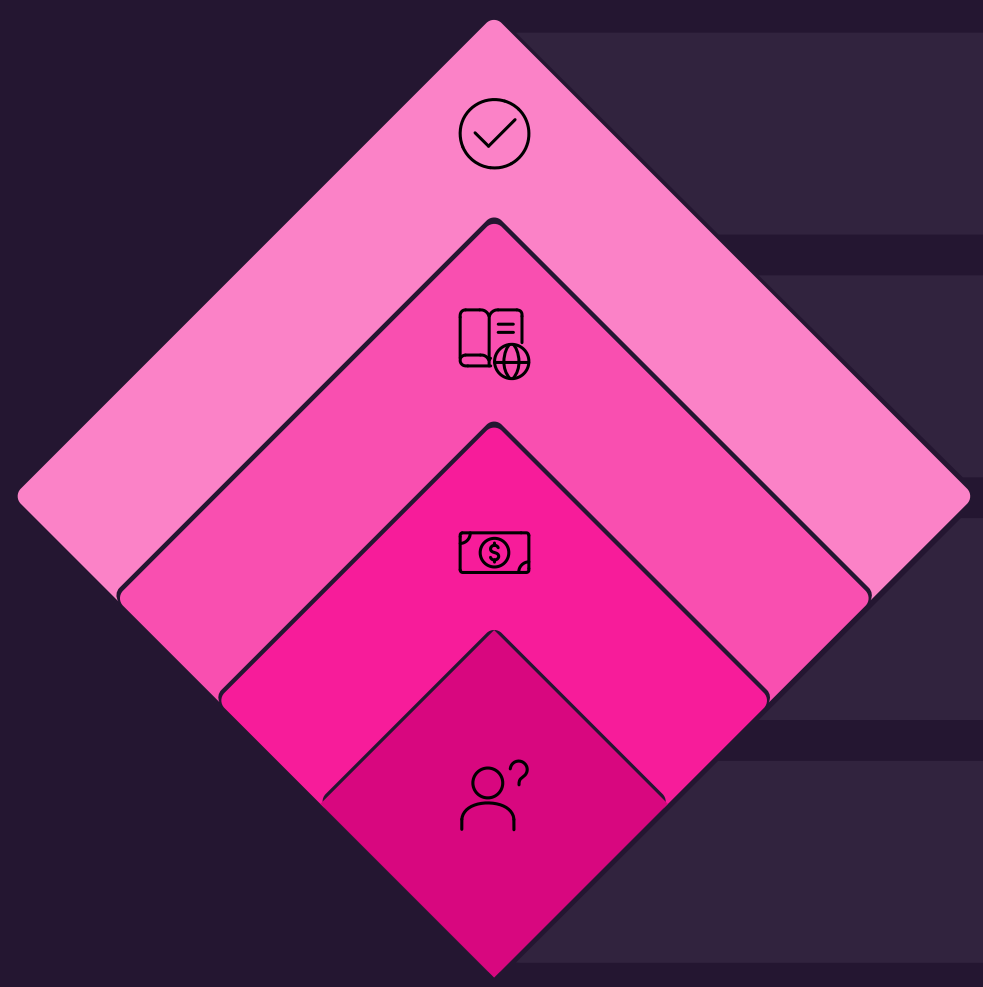
✅ Ventajas

- Fácil de interpretar y visualizar
- No requiere escalado de características
- Maneja datos numéricos y categóricos
- Captura relaciones no lineales

⚠️ Limitaciones

- Propenso al **sobreajuste (overfitting)**
- Inestable: pequeños cambios en datos generan árboles muy distintos
- Fronteras de decisión axis-aligned (paralelas a los ejes)
- Sesgado hacia clases mayoritarias

ⓘ 🌳 **Parámetros clave:** *max_depth* (profundidad máxima), *min_samples_split* (mínimo de muestras para dividir), *min_samples_leaf* (mínimo de muestras por hoja). Regularizar estos parámetros es esencial para evitar overfitting.



Hojas
Aprobar / Rechazar

Nodo interno 2
¿Historial = bueno?

Nodo interno 1
¿Salario > 50k?

Nodo raíz
¿Edad > 30?

El diagrama ilustra cómo un árbol de decisión realiza divisiones sucesivas del espacio de características. Cada pregunta divide los datos en dos grupos más puros hasta llegar a las hojas, donde se asigna la clase mayoritaria de ese grupo.

Made with GAMMA

Modelos 3 y 4: Random Forest y Máquinas de Soporte Vectorial

🌲 Random Forest Classifier

El **Random Forest** es un método de **ensamble** que combina múltiples árboles de decisión para mejorar la precisión y reducir el sobreajuste. Cada árbol se entrena con una muestra aleatoria del dataset (bootstrapping) y con un subconjunto aleatorio de características en cada división.

La predicción final es la **votación mayoritaria** entre todos los árboles del bosque.

- **Robusto:** menos sensible al ruido y outliers que un árbol individual
- **Paralelizable:** los árboles se entrenan de forma independiente
- **Importancia de variables:** mide qué características son más relevantes
- **Pocos hiperparámetros:** principalmente *n_estimators* y *max_depth*

⚡ Support Vector Machines (SVM)

Las **Máquinas de Soporte Vectorial** buscan el **hiperplano óptimo** que separa las clases con el mayor margen posible. Los puntos más cercanos al hiperplano se llaman *vectores de soporte* y son los únicos que determinan la frontera de decisión.

Mediante el **kernel trick**, las SVM pueden crear fronteras no lineales mapeando los datos a espacios de mayor dimensionalidad sin calcularlo explícitamente.

- **Kernels comunes:** lineal, polinómico, RBF (radial basis function)
- **Parámetro C:** controla el trade-off entre margen y errores de entrenamiento
- **Efectivas en alta dimensión:** funcionan bien con muchas características
- **Sensibles al escalado:** requieren normalización de las variables

🔬 **Random Forest vs SVM:** Random Forest es más robusto con datos ruidosos y no requiere escalado. SVM puede ser más preciso en problemas con pocas muestras y muchas dimensiones, pero es más costoso computacionalmente.

Made with GAMMA

Conclusiones y Ejemplos del Mundo Real

La clasificación es una herramienta poderosa con aplicaciones en prácticamente todos los sectores. Elegir el algoritmo adecuado depende del problema, los datos disponibles y los requisitos de interpretabilidad y rendimiento.

📋 Resumen de algoritmos

01

Regresión Logística

Simple, interpretable, ideal como baseline y cuando necesitas probabilidades calibradas.

02

Árboles de Decisión

Fáciles de explicar, capturan no linealidades, pero propensos a sobreajuste sin regularización.

03

Random Forest

Robusto, preciso, maneja bien el desbalanceo. Excelente opción por defecto en muchos problemas.

04

SVM

Potente en espacios de alta dimensión, efectivo con pocas muestras. Requiere escalado y ajuste cuidadoso.

🌍 Ejemplos de aplicación

Detección de Spam

Clasificación binaria: cada correo es etiquetado como **spam o legítimo** analizando contenido, remitente y metadatos. Uno de los primeros éxitos comerciales del ML.

Diagnóstico Médico

Clasificación binaria o multiclase para detectar enfermedades (cáncer, diabetes, COVID-19) a partir de pruebas clínicas e imágenes. El **recall alto** es crítico aquí.

Detección de Fraude

Clasificación binaria con datos **muy desbalanceados** (menos del 1 % de transacciones son fraudulentas). Requiere técnicas especiales de muestreo y métricas como AUC-ROC.

Análisis de Sentimiento

Clasificación multiclase que asigna a un texto una etiqueta emocional: **positivo, neutro o negativo**. Fundamental en monitorización de redes sociales y atención al cliente.

🎯 **Conclusión final:** La clasificación permite automatizar decisiones complejas a escala. La clave del éxito está en entender bien el problema, preparar los datos correctamente (especialmente el balanceo), elegir métricas adecuadas y seleccionar el algoritmo que mejor se adapte al contexto. No existe un algoritmo universalmente superior: el mejor modelo es el que mejor resuelve *tu* problema concreto.

Made with GAMMA